

Function Pointer

A **function pointer** in C is a pointer that stores the address of a function. Just like pointers to variables, a function pointer allows you to reference and call a function indirectly. Function pointers are particularly useful for implementing callback mechanisms, dynamic function calls, and creating flexible and modular programs.

Key Features of Function Pointers:

1. **Stores Function Address:** A function pointer holds the memory address of a function.
2. **Matches Function Signature:** The type of the function pointer must match the signature (return type and parameters) of the function it points to.
3. **Calls Function Indirectly:** You can invoke the function using the function pointer.

General Format: `return_type (*pointer_name)(parameter_list);`

`int (*p)(int, int);`

Here, p is a pointer to a function that takes two int arguments and returns an int.

`int (*p)();`

The declaration `int (*p)();` means that p is a **pointer to a function** that returns an int. This is used when you want to store the address of a function and later call that function through the pointer.

```
#include <stdio.h>
// Function to be pointed to
int myFunction() {
    return 25;
}
int main() {
    // Declare a pointer to a function returning int
    int (*p)();
    // Assign the function's address to the pointer
    p = myFunction;
    // Call the function using the pointer
    printf("Function returned: %d\n", p()); // Using p() directly
    printf("Function returned: %d\n", (*p)()); // Using (*p)() explicitly
    return 0;
}
//Output
Function returned: 25
Function returned: 25
```

//Advanced Example: Using a Function Pointer Array

```
#include <stdio.h>
```

```
// Some example functions
```

```
int add() { return 5 + 3; }
```

```
int subtract() { return 5 - 3; }
```

```
int multiply() { return 5 * 3; }
```

```
int main() {  
    // Array of function pointers  
    int (*operations[3])() = {add, subtract, multiply};  
  
    // Call functions through the pointer array  
    printf("Addition: %d\n", operations[0]());  
    printf("Subtraction: %d\n", operations[1]());  
    printf("Multiplication: %d\n", operations[2]());  
  
    return 0;  
}
```

//Output:

Addition: 8

Subtraction: 2

Multiplication: 15

This shows how function pointers allow flexible, modular, and dynamic function calls.

Explain **int (*p)[5];**

int (*p)[5] is a pointer to an array of 5 integers. Instead of pointing to a single integer, p points to the first element of an array containing exactly 5 integers.

```
#include <stdio.h>
```

```
int main() {  
    int arr[5] = {1, 2, 3, 4, 5}; // Array of 5 integers  
    int (*p)[5] = &arr;          // Pointer to the array of 5 integers  
    // Access elements of the array using the pointer  
    printf("First element: %d\n", (*p)[0]);  
    printf("Second element: %d\n", (*p)[1]);  
    printf("Third element: %d\n", (*p)[2]);  
    // Modify an element through the pointer.  
    //To access or modify the array, dereference the pointer (*p) and use array  
    indexing.  
    (*p)[3] = 10; // Changing the fourth element to 10  
    printf("Modified fourth element: %d\n", (*p)[3]);  
    return 0;  
}
```

//Output:

First element: 1

Second element: 2

Third element: 3

Modified fourth element: 10

When passing a 2D array or higher-dimensional array to a function, using a pointer to an array ensures the function understands the dimensions of the array.

```
void printArray(int (*arr)[5], int rows) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 5; j++) {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }
}
```

```
int main() {
    int matrix[3][5] = {{1, 2, 3, 4, 5},
                        {6, 7, 8, 9, 10},
                        {11, 12, 13, 14, 15}};
    printArray(matrix, 3);
    return 0;
}
```

Here, `int (*arr)[5]` explicitly tells the compiler that each row contains 5 integers.

With `int (*p)[5]`, you ensure the pointer always points to an array of exactly 5 integers. This can prevent accidental misuse where a generic `int*` might lead to accessing out-of-bounds memory.

A pointer to an array allows easy manipulation of array blocks as a unit. Incrementing the pointer moves it by the size of the entire array (e.g., `5 * sizeof(int)`), not just one element.

```
int arr[3][5] = {{1, 2, 3, 4, 5}, {6, 7, 8, 9, 10}, {11, 12, 13, 14, 15}};
```

```
int (*p)[5] = arr;
```

```
p++; // Moves the pointer to the next row of 5 integers
```

```
printf("First element of the second row: %d\n", (*p)[0]); // Outputs 6
```

Using `int (*p)[5]` clearly conveys the developer's intent: *This pointer is meant to work with arrays of 5 integers*. This can improve code readability and maintainability in projects where array sizes are critical.